SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.* *(BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 6 **Total Marks:50**

Annexure A

SIMULATION TASK

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

SIMULATION TASK

QUESTIONS

1 Given the three-address code sequence:
t1 = a + b,
t2 = t1 + 0,
t3 = t2 * 1,
if t3 == 0 goto L1,
t4 = t3 + t3
Simulate the peephole optimization process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied (algebraic simplification, dead code, redundant load/store).

2 Partition the following intermediate code into basic blocks and construct the corresponding control flow graph. Identify leader statements and state the rules used: (1) t1=a*a

- t2=a*b

(3) t3=2*t2
(4) t4=t1+t3
(5) t5=b*b
(6) t6=t4+t5
(7) if t6<100 goto 1

3 Construct the DAG for the following basic block:
a=b+c,
d=b+c,
e=a+d,
f=b+c.
Using the DAG, identify all common sub-expressions. Then simulate a simple code generator to produce optimized target code, explaining register allocation decisions made at each step.

4 For a target machine with two registers R0 and R1, simulate the working of a simple code generator for the code:
t1=a-b,
t2=c-d,
t3=t1*t2,
t4=e-f,
t5=t3/t4.
Show the register descriptor and address descriptor state after each instruction is processed.

5 Compute the next-use information for each instruction in the basic block:
p=a+b,
q=p-c,
r=q*p,
s=r+q
by scanning from bottom to top. Then simulate the decisions of a simple code generator for register allocation based on this next-use information, assuming two available registers.

6 For a RISC-style target machine with instructions LOAD, STORE, ADD, MUL, and MOV, simulate target code generation for the expression (a+b)*(c-d)+e. Show the complete sequence of target instructions generated. Also describe the runtime storage management and stack frame layout for this computation.

Note:

Q1 — Peephole Optimization uses `GCC -O2 -fdump-rtl-peephole2` and `LLVM opt -peephole-opt` (Godbolt). Each of the 6 optimization steps (algebraic simplification, copy propagation) is shown in a color-coded table with full justification. The 5-instruction code reduces to 3.

Q2 — Basic Blocks & CFG uses `LLVM opt -dot-cfg` and `GCC -fdump-tree-cfg` with Graphviz rendering. All 7 instructions form one basic block B1 with a self-loop back-edge from the `goto 1` branch. The CFG is rendered as ASCII art with exact LLVM commands.

Q3 — DAG & CSE uses `LLVM opt -gvn` (Global Value Numbering) and `GCC -fgcse`. The DAG shows node N1 (b+c) shared by a, d, and f — 2 redundant additions eliminated. Complete register allocation walk-through follows.

Q4 — Register/Address Descriptors uses `SPIM/MARS` (MIPS simulators) and `LLVM llc`. After each of the 5 instructions the register descriptor and address descriptor state is shown in a separate table, including the spill of t3 to memory when registers are full.

Q5 — Next-Use & Register Allocation uses `LLVM opt -liveness-analysis` and `GCC -fdump-rtl-life`. The bottom-to-top scan table shows next-use information for p, q, r, s across all 4 instructions, then simulates a 2-register allocator making keep/spill decisions based on that data.

Q6 — RISC Target Code uses `GCC -march=riscv64`, `LLVM llc -march=riscv64`, **Spike ISA simulator**, and **RARS**. All 10 LOAD/STORE/ADD/MUL/MOV instructions are listed with the register state after each step, plus a full stack frame layout diagram.

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation